

KPIs Considered for the V0 → V1 Refactor

Agentic-RAG POC — backend + frontend performance and confidence-rollup refactor

Ahead of the refactor, two read-only reviewer agents (backend-perf-review, frontend-perf-review) audited the codebase and filed severity-tagged findings; a separate open triage finding (3.3) flagged a confidence-scoring correctness gap. The KPIs below are the concrete, measurable signals used to decide what to fix, and to verify each fix afterward — pulled directly from those review reports, the unit/integration test suite, the k6 load test, and live post-refactor measurements (API usage fields, server timing, and a real headless-browser DOM test).

1. Backend Performance KPIs

KPI	Baseline (V0)	Target	V1 Outcome
Prompt-cache utilization cache_read/cache_creation_input_tokens per request	0 — no cache_control anywhere	Non-zero, growing across loop iterations	Confirmed live via response.usage: 0 → 2647 (read), 0 → 314 (creation) across 2 iterations of a real query
Cold embedder load time on first request time-to-first-response after process start	~15.9s model load + 0.57s first inference, paid by whichever user's request happened to be first	Eliminated from the user-facing request path	Moved to a startup hook; /api/health responded in 49ms immediately after "startup complete"
Embedding calls per multi-search turn embed_query call count when a turn issues N search_corpus calls	N sequential embed_query calls (N+1- style)	1 batched embed_texts call per turn	Verified via dedicated unit test: exactly 1 batched call for a 2-block turn
Batch vs. sequential embedding latency 3-item query batch	~57ms summed (3 sequential calls)	Lower via one batched call	39.7ms measured for the equivalent 3-item batch
submit_structured_answer schema build cost Pydantic model_json_schema() calls per request	Rebuilt from scratch on every request (static schema)	Computed once, reused	@lru_cache(maxsize=1) applied

2. Confidence-Scoring Correctness KPI

KPI	Baseline (V0)	Target	V1 Outcome
Overall-confidence fidelity to actual evidence strength TRIAGE_REPORT finding 3.3	Strict weakest-link <code>min()</code> : any single Low claim caps the whole answer, regardless of how many sibling claims are strong (a 13-	Supermajority ($\geq 2/3$ of claims) threshold reflects the real distribution of evidence strength	Implemented and hand-verified against 5 required cases + 3 unit tests. Live re-run of the triage question twice: once resolved Moderate (supermajority met), once resolved Low (a genuine majority- weak claim decomposition) — fix

KPI	Baseline (V0)	Target	V1 Outcome
	claim, 4-domain answer with 9 strong claims still reported "Low")		behaves correctly in both directions, but doesn't guarantee escaping "Low" on every run since claim decomposition is model-non-deterministic

3. Frontend Performance KPIs

KPI	Baseline (V0)	Target	V1 Outcome
DOM node growth per session count of .turn elements in #results-area	Unbounded – grows linearly with every question asked, never released	Fixed ceiling	Capped at exactly 20; verified live in headless Chrome: after 22 submissions, DOM held exactly 20 .turn elements with the oldest 2 genuinely removed (not hidden)
Event listener count (citation pills) click listeners attached across a session	0(total citations rendered across the whole session) – one closure per pill, never released	O(1), delegated to a stable container	Single delegated listener on #results-area; verified live that clicks route correctly via event.target.closest()
Eager DOM construction for collapsed content citation quote blocks + retrieval-log	Built unconditionally at render time for every citation/turn, even if never expanded	Deferred to first expand, cached after	Verified live: quote/log DOM absent before first click/open, correctly built (with right content) on first interaction, toggles freely afterward

4. Reliability / Load KPIs

KPI	Baseline (V0, Run 1)	Target	V1 Outcome (Run 2)
k6 p95 request latency POST /api/query under ramping load, 0 → 6 VUs	35.29s	< 60s threshold	61s — threshold breached. Flagged as likely external API/proxy latency drift, not a code regression: two unrelated manual queries in the same session independently took 63–65s each on unchanged model-call-dominated latency
k6 HTTP request-failure rate	0%	< 5% threshold	0%
k6 RAG schema-validity error rate	0%	< 5% threshold	0%
Unit/integration test pass rate pytest tests -q --ignore=tests/load	All passing pre-refactor	100% pass, including new tests	44 passed, 3 skipped (pre-existing live-server-only smoke tests, unchanged)

5. Reviewed but Not Actioned (confirmed negligible — considered, then deliberately deferred)

KPI considered	Why it was not actioned
Session-level search_corpus query dedup/cache hit rate	Lower priority than prompt caching, which already reduces the cost of a repeated search from the other direction; a second dedup layer would be speculative complexity at this corpus scale.
Allowlist re-parse cost per request (clear_cache() before every read)	Measured at 0.24ms/call — negligible against real model-call latency.
Tool-schema (search_corpus) construction cost per request	Measured at ~1.46ms — negligible.
Chunking overhead / chunk size vs. retrieval quality	Ingestion-time only, not a per-request cost; out of this refactor's scope.
Double-submit guard robustness, JSON.stringify-per-render cost, innerHTML-then-requery pattern (frontend P3s)	Reviewer explicitly flagged these as "no action needed" / optional polish only.

Generated from docs/PERF_REVIEW_BACKEND.md, docs/PERF_REVIEW_FRONTEND.md, docs/TRIAGE_REPORT.md (finding 3.3), docs/LOAD_TEST_REPORT.md, and docs/REFACTOR_V1_REPORT.md — Agentic-RAG POC, V0 → V1 refactor.